

```
#!/usr/bin/env python3
```

```
"""
```

Project #5 – Folder and File Creation in Python (Timestamped Variant)

Author: Ahmed, Irfan

OS: macOS (Apple Silicon)

Description:

Prompts for a BASE folder name, appends a timestamp (YYYY-MM-DD_HH-MM-SS), removes any existing folder with that exact name (unlikely but safe), recreates it, and writes 100 random integers [0, 1000] to numbers100.txt. Includes a verification step to ensure rubric compliance.

Rubric mapping:

- Prompt for folder name → asks for base name; resulting folder uses timestamp.
- Existing folder removal → if exact timestamped name exists, it is removed.
- 100 numbers in numbers100.txt inside the created folder → enforced & verified.
- Professional comments include name and OS.

Note:

Uses pathlib/shutil/random.SystemRandom() for OS-backed operations.

```
"""
```

```
from __future__ import annotations
```

```
import shutil
```

```
from pathlib import Path
```

```
from datetime import datetime
```

```
import random
```

```
import sys
```

```
INVALID_NAME_CHARS = set('<>:"/\|?*')
```

```
def prompt_base_name() -> str:
```

```
    while True:
```

```
        name = input("Enter BASE folder name (timestamp will be added automatically): ").strip()
```

```
        if not name:
```

```
            print("Folder name cannot be empty. Try again.\n"); continue
```

```
        if any(ch in INVALID_NAME_CHARS for ch in name):
```

```
            print('Avoid these characters: <>:"/\|?*\\n'); continue
```

```
        return name
```

```
def make_timestamped_name(base: str) -> str:
```

```
    stamp = datetime.now().strftime('%Y-%m-%d_%H-%M-%S')
```

```
    return f"{base}_{stamp}"
```

```
def recreate_folder(base_dir: Path, folder_name: str) -> Path:
```

```
    target = base_dir / folder_name
```

```
    if target.exists():
```

```
        if target.is_dir():
```

```
            print(f"Folder '{folder_name}' already exists. Removing it...")
```

```
            shutil.rmtree(target)
```

```
        else:
```

```
            print(f"A file named '{folder_name}' exists. Deleting it...")
```

```

        target.unlink()
    print(f"Creating folder '{folder_name}'...")
    target.mkdir(parents=True, exist_ok=False)
    return target

def write_random_numbers(dest_dir: Path,
                        count: int = 100,
                        low: int = 0,
                        high: int = 1000,
                        filename: str = "numbers100.txt") -> Path:
    rng = random.SystemRandom()
    nums = (str(rng.randrange(low, high + 1)) for _ in range(count))
    out_path = dest_dir / filename
    out_path.write_text("\n".join(nums) + "\n", encoding="utf-8")
    return out_path

def verify_output(folder: Path, filename: str = "numbers100.txt") -> None:
    """Deep verification: existence, line count, value range, formatting."""
    txt = folder / filename
    if not folder.exists() or not folder.is_dir():
        raise AssertionError(f"Folder not found: {folder}")
    if not txt.exists():
        raise AssertionError(f"File not found: {txt}")

    content = txt.read_text(encoding="utf-8")
    # Ensure final newline exists
    if not content.endswith("\n"):
        raise AssertionError("numbers100.txt is missing a trailing newline")

    lines = content.splitlines()
    if len(lines) != 100:
        raise AssertionError(f"Expected 100 lines, found {len(lines)}")

    # Validate each number is an int within range
    for i, line in enumerate(lines, 1):
        try:
            val = int(line.strip())
        except ValueError as exc:
            raise AssertionError(f"Line {i} is not an integer: {line!r}") from exc
        if not (0 <= val <= 1000):
            raise AssertionError(f"Line {i} out of range [0,1000]: {val}")

    print("Verification passed: 100 integers in [0,1000], correct formatting, trailing newline present.")

def main() -> int:
    print("=== Project #5 (Timestamped): Folder and File Creation in Python ===")
    cwd = Path.cwd()
    print(f"Current working directory: {cwd}")

    base = prompt_base_name()
    folder_name = make_timestamped_name(base)

```

```
try:
    target_dir = recreate_folder(cwd, folder_name)
    out_file = write_random_numbers(target_dir)
    verify_output(target_dir)
except (PermissionError, OSError) as e:
    print(f"OS error: {e}", file=sys.stderr); return 1
except AssertionError as e:
    print(f"Verification failed: {e}", file=sys.stderr); return 2

print(f"Success! Wrote 100 random numbers to: {out_file}")
print(f"Created folder: {target_dir}")
return 0

if __name__ == "__main__":
    raise SystemExit(main())
```